

QueryDSL

type-safe Query

김영한

목차

- 문제
- JPA Query
- QueryDSL?
- QueryDSL-JPA
- 기능 살펴보기

문제

긴급 요구사항

- 검색 조건 추가
- 나이
- 이름
- 퇴근시간 10분전...

기능 추가



이정도는 발로 만들어 주지

쿼리 추가

String sql =

“select * from member” +

“where name like ?” +

“and age between ? and ?”

컴파일 완료



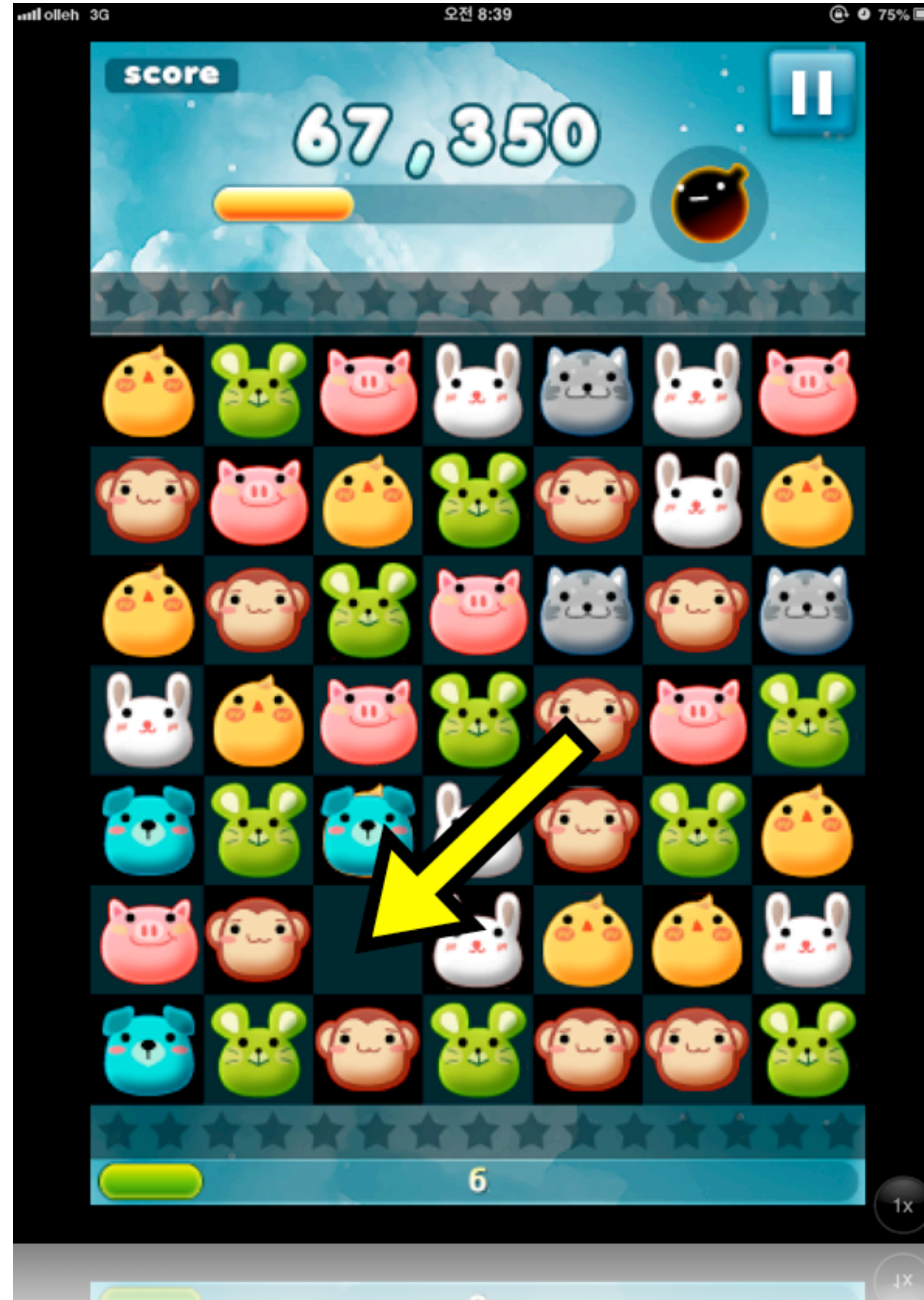
배포 완료



퇴근 완료



버그 발생



리콜 완료??



그럴리가 없어
분명 빌드는 성공했어!

버그

String sql =

“select * from member” +

“where name like ?” +

“and age between ? and ?”

[문자를 합치면]

“select * from **memberwhere** name like ?and age
between ? and ?”

QUERY의 문제점

- QUERY는 문자, Type-check 불가능
- 실행하기 전까지 작동여부 확인 불가

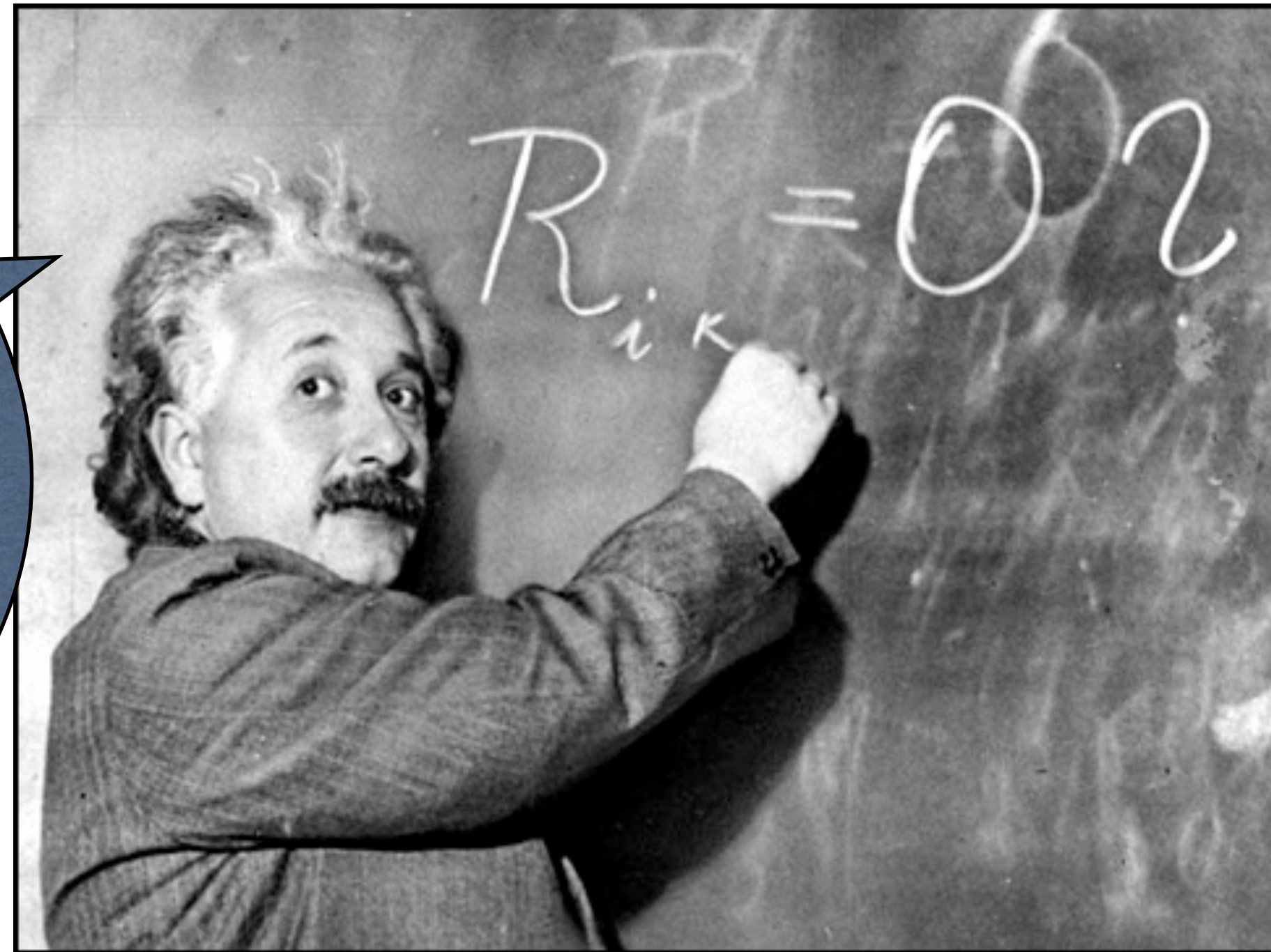
에러는 크게 2가지

- 컴파일 에러 (좋은 에러)
- 런타임 에러 (나쁜 에러)

도메인 교수?

전 SQL을 빨리 작성하기 위해 컬럼명 정도는
모두 외운답니다.

`SELECT NAME, AGE ..`



SQL, JPQL

- 만약 SQL이 클래스처럼 타입이 있고 자바 코드로 작성 할 수 있다면?
- type-safe

Type-safe

- 컴파일시 에러 체크 가능
- Code-assistant x 100!!!
- CTRL + SPACE + . (DOT)

QueryDSL

- 쿼리를 Java로 type-safe하게 개발할 수 있게 지원하는 프레임워크
- 주로 JPA 쿼리(JPQL)에 사용

JPA Query

java ORM

질문 : 사람을 찾아보자

- 20~40살
- 성 = 김씨
- 나이 많은 순서
- 3명을 출력하라.

회원

@Entity

public class Member {

 @Id @GeneratedValue

 private Long id;

 private String name;

 private int age;

 ...
}

회원 Table

```
create table Member (  
    id bigint auto primary key,  
    age integer not null,  
    name varchar(255)  
)
```

JPA에서 QUERY 방법은 크게 3가지

- 1. JPQL(HQL)
- 2. Criteria API
- 3. MetaModel Criteria API(type-safe)

1. JPQL(HQL)

@Test

public void jpql() {

String query =

"select m from Member m " +
"where m.age between 20 and 40 " +
" and m.name like '김%' " +
"order by m.age desc";

List<Member> resultList =

entityManager.createQuery(query, Member.class)
.setMaxResults(3).getResultList();

}

1. JPQL(HQL)

- 장점 : SQL QUERY와 비슷해서 금방 익숙해짐
- 단점 : type-safe 아님
동적쿼리 생성이 어려움

2. Criteria API

@Test

public void jpaCriteriaQuery() {

```
CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<Member> cq = cb.createQuery(Member.class);
Root<Member> root = cq.from(Member.class);
```

```
Path<Integer> age = root.get("age");
Predicate between = cb.between(age, 20, 40);
```

```
Path<String> path = root.get("name");
Predicate like = cb.like(path, "김%");
```

```
CriteriaQuery<Member> query = cq.where( cb.and(between, like) );
query.orderBy(cb.desc(age));
```

```
List<Member> resultList =
    entityManager.createQuery(query).setMaxResults(3).getResultList();
```

```
}
```

2. Criteria API

```
select id, age, name  
from MEMBER  
where age between 20 and 40 and name like '김%'  
order by age desc limit 3
```

2. Criteria API

- 장점 : 동적쿼리 생성이 쉬움??
- 단점
 - 1. type-safe 아님
 - 2. 너무 너무 너무 복잡함
 - 3. 알아야 할게 너무 많음

3. MetaModel Criteria API (type-safe)

- `root.get("age") -> root.get(Member_.age)`
- Criteria API + MetaModel
- Criteria API와 거의 동일
- type-safe
- 복잡하긴 마찬가지

이건 아니야--;



timowest

QueryDSL

QueryDSL 분석

- Domain(도메인)
- Specific(특화)
- Language(언어)

DSL

- 도메인 + 특화 + 언어
- 특정한 도메인에 초점을 맞춘 제한적인 표현력을 가진 컴퓨터 프로그래밍 언어
- 특징 : 단순, 간결, 유창

QueryDSL

- 쿼리 + 도메인 + 특화 + 언어
- 쿼리에 특화된 프로그래밍 언어
- 단순, 간결, 유창
- 다양한 저장소 쿼리 기능 통합

데이터 쿼리 기능 추상화

QueryDSL API

[Persistence]

JPA, SQL, JAVA Collections, RDF, Lucene, Hibernate Search,
Mongo Db

QueryDSL?

- JPA, MongoDB, SQL 같은 기술들을 위해 type-safe SQL을 만드는 프레임워크

Type-safe Query Type 생성



코드생성기

- APT : Annotation Processing Tool
 - @Entity

QueryDSL-JPA

Querydsl-JPA

- Querydsl은 JPA 쿼리(JPQL)을 typesafe 하게 작성하는데 많이 사용됨
- 너를 위해 만들었다!

질문 : 사람을 찾아보자

- 20~40살
- 성 = 김씨
- 나이 많은 순서
- 3명을 출력하라.

회원 Table

```
create table Member (  
    id bigint auto primary key,  
    age integer not null,  
    name varchar(255)  
)
```

회원

@Entity

public class Member {

 @Id @GeneratedValue

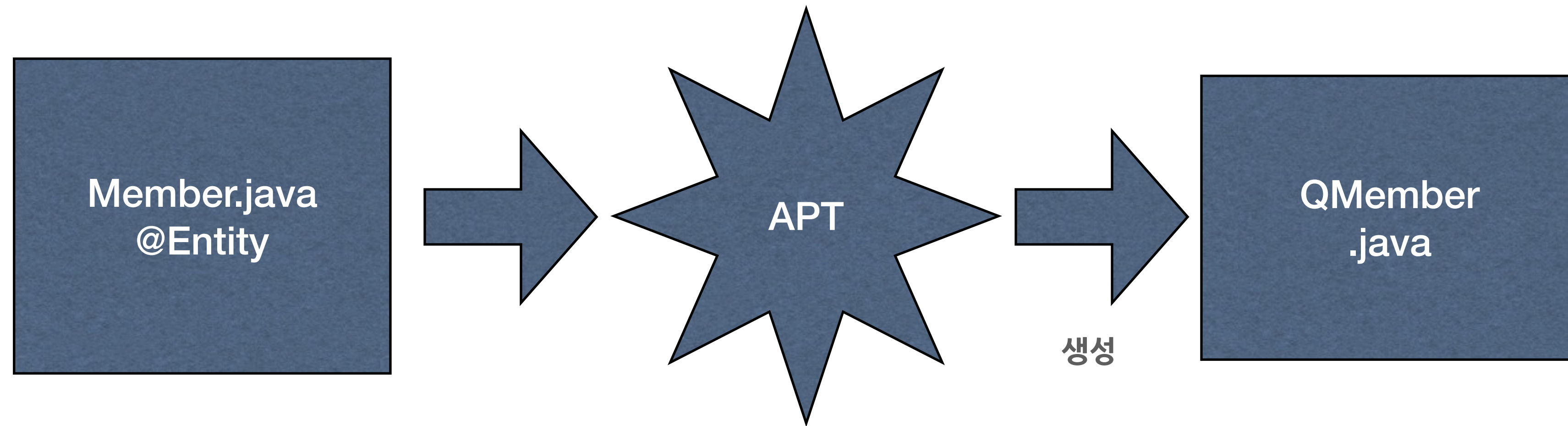
 private Long id;

 private String name;

 private int age;

 ...
}

QueryDSL-JPA



자동생성된 회원Query

```
@Generated("com.mysema.query.codegen.EntitySerializer")
public class QMember extends EntityPathBase<Member> {

    public final NumberPath<Long> id = createNumber("id", Long.class);
    public final NumberPath<Integer> age = createNumber("age", Integer.class);
    public final StringPath name = createString("name");

    public static final QMember member = new QMember("member");

    ...
}
```

Querydsl-JPA

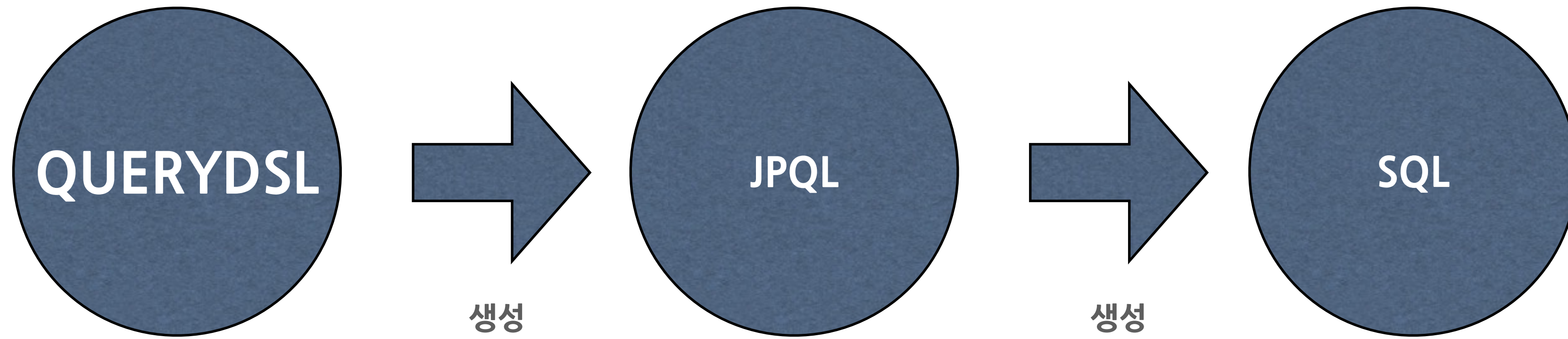
```
JPAQueryFactory query = new JPAQueryFactory (entityManager);  
QMember m = QMember.member;
```

```
List<Member> list = query  
    .select(m)  
    .from(m)  
    .where(  
        m.age.between(20, 40).and(m.name.like("김%"))  
    )  
    .orderBy(m.age.desc())  
    .limit(3)  
    .fetch(m);
```

생성된 쿼리

```
select id, age, name  
from MEMBER  
where age between 20 and 40  
and name like '김%'  
order by age desc  
limit 3
```

작동 방식



Querydsl-JPA

- 장점
 - type-safe
 - 단순함
 - 쉬움
- 단점
 - Q코드 생성을 위한 APT를 설정해야함

기능 살펴보기

QueryDSL-JPA

구성

Query

ex) from, where, join

Path

ex) QMember, QMember.name

Expression

ex) name.eq, name.gt

기능 살펴보기

- from
- innerJoin, join, leftJoin, fullJoin, on
- where (and, or, allOf, anyOf)
- groupBy
- having
- orderBy (desc, asc)
- limit, offset, restrict(limit + offset) (Paging)

기능 살펴보기

- list
- iterate
- count
- fetch(): 목록 조회
- fetchOne(): 단건 조회

단순 쿼리

```
QMember m = QMember.member;
```

```
List<Member> results = query  
    .select(m)  
    .from(m)  
    .where(m.name.startsWith("김").and(m.age.between(20, 40)))  
    .fetch(m);
```

동적 쿼리

```
String firstName = "김"; int min=20,max=40;
```

```
BooleanBuilder builder = new BooleanBuilder();  
if (StringUtils.hasText(str))  
    builder.and(m.name.startsWith(firstName));
```

```
if (min != 0 && max != 0)  
    builder.and(m.age.between(min, max));
```

```
List<Member> results = query  
    .select(m)  
    .from(m)  
    .where(builder)  
    .fetch(m);
```

조인 쿼리

```
QMember m = QMember.member;  
QMemberCard mc = QMemberCard.memberCard;
```

```
List<Member> list = query  
    .select(m)  
    .from(m).join(m.memberCards, mc)  
    .fetch(m);
```

페이징, 정렬

```
List<Member> result = queryFactory
    .selectFrom(member)
    .orderBy(member.username.desc())
    .offset(1) //0부터 시작(zero index)
    .limit(2) //최대 2건 조회
    .fetch();
```

SpringDataJPA + Querydsl

- SpringData프로젝트의 약점은 조회
- Querydsl로 복잡한 조회 기능 보완
 - 복잡한 쿼리
 - 동적 쿼리
- 단순한 경우 : SpringDataJPA
- 복잡한 경우 : Querydsl직접 사용

QueryDSL 결론

- 한번 써보면 돌아 올 수 없음
- 감동의 컴파일 에러
- 감동의 IDE 지원, ctrl+space, Code-assistant
- JPQL로 해결하기 어려운 복잡한 쿼리는 네이티브 SQL 쿼리 사용 (JdbcTemplate, MyBatis)